Agent 体系详解 — 11 个 Agent 全景

Agent 类型系统

> 源文件: `src/agents/types.ts`

```typescript
// src/agents/types.ts

/**
 * Agent mode determines UI model selection behavior:
 * - "primary": Respects user's UI-selected model (sisyphus, atlas)
 * - "subagent": Uses own fallback chain, ignores UI selection (oracle, explore, etc.)
 * - "all": Available in both contexts (OpenCode compatibility)
 */
export type AgentMode = "primary" | "subagent" | "all"

/**
 * Agent factory function with static mode property.
 * Mode is exposed as static property for pre-instantiation access.
 */
export type AgentFactory = ((model: string) => AgentConfig) & {
  mode: AgentMode
}

/** Agent category for grouping in Sisyphus prompt sections */
export type AgentCategory = "exploration" | "specialist" | "advisor" | "utility"

/** Cost classification for Tool Selection table */
export type AgentCost = "FREE" | "CHEAP" | "EXPENSIVE"

/** Delegation trigger for Sisyphus prompt's Delegation Table */
export interface DelegationTrigger {
  domain: string   // e.g., "Frontend UI/UX"
  trigger: string  // e.g., "Visual changes only..."
}

/**
 * Metadata for generating Sisyphus prompt sections dynamically.
 * This allows adding/removing agents without manually updating the Sisyphus prompt.
 */
export interface AgentPromptMetadata {
  category: AgentCategory
  cost: AgentCost
  triggers: DelegationTrigger[]
  useWhen?: string[]
  avoidWhen?: string[]
  dedicatedSection?: string
  promptAlias?: string
  keyTrigger?: string
```

```
}

export type BuiltinAgentName =
  | "sisyphus" | "hephaestus" | "oracle" | "librarian"
  | "explore" | "multimodal-looker" | "metis" | "momus" | "atlas"

export type AgentOverrideConfig = Partial<AgentConfig> & {
  prompt_append?: string
  variant?: string
  fallback_models?: string | string[]
}
```

模式	含义	代表 Agent
`primary`	尊重用户 UI 选择的模型	—
`subagent`	使用自有 fallback 链，忽略 UI 选择	Oracle, Explore, Librarian, Metis, Momus
`all`	在两种上下文中均可用	Sisyphus, Hephaestus, Atlas

Agent 注册中心

> 源文件: `src/agents/builtin-agents.ts`

```ts
// src/agents/builtin-agents.ts — Agent 工厂注册表
const agentSources: Record<BuiltinAgentName, AgentSource> = {
  sisyphus: createSisyphusAgent,
  hephaestus: createHephaestusAgent,
  oracle: createOracleAgent,
  librarian: createLibrarianAgent,
  explore: createExploreAgent,
  "multimodal-looker": createMultimodalLookerAgent,
  metis: createMetisAgent,
  momus: createMomusAgent,
  atlas: createAtlasAgent as AgentFactory,  // Atlas 需要 OrchestratorContext
}

/** Agent 元数据注册表，用于构建 Sisyphus 的动态 prompt */
const agentMetadata: Partial<Record<BuiltinAgentName, AgentPromptMetadata>> = {
  oracle: ORACLE_PROMPT_METADATA,
  librarian: LIBRARIAN_PROMPT_METADATA,
  explore: EXPLORE_PROMPT_METADATA,
  "multimodal-looker": MULTIMODAL_LOOKER_PROMPT_METADATA,
  metis: metisPromptMetadata,
  momus: momusPromptMetadata,
  atlas: atlasPromptMetadata,
}
```

1. Sisyphus — 主编排器

属性	值
源文件	`src/agents/sisyphus.ts` (598 行)
模式	`all`
模型	Claude Opus 4-6 (max) → Kimi K2.5 → GLM-5 → Big Pickle
Token	maxTokens: 64000, thinking budgetTokens: 32000
角色	顶层 AI 编排器，"SF 湾区高级工程师"

核心能力

- 解析隐含需求
- 适应代码库成熟度（disciplined vs chaotic）
- 委派专业工作给正确的子代理
- 并行执行最大化吞吐
- 遵循用户指令，不主动开始实现

四阶段工作流

Phase 0 — Intent Gate（每条消息必做）

```
Step 0: 意图提取
  ├── "explain X" → 研究/理解 → explore/librarian → 综合 → 回答
  ├── "implement X" → 实现（显式）→ 规划 → 委派或执行
  ├── "look into X" → 调查 → explore → 报告发现
  ├── "what do you think?" → 评估 → 提议 → 等待确认
  ├── "error X / Y is broken" → 修复 → 诊断 → 最小修复
  └── "refactor / improve" → 开放性 → 先评估代码库 → 提议方案

Step 1: 分类请求
  ├── Trivial → 直接工具（除非 Key Trigger）
  ├── Explicit → 直接执行
  ├── Exploratory → 并行 explore(1-3) + 工具
  ├── Open-ended → 先评估代码库
  └── Ambiguous → 提一个澄清问题

Step 2: 歧义检查
  ├── 单一解释 → 继续
  ├── 多解释，类似工作量 → 用合理默认继续
  └── 多解释，2x+ 工作量差异 → 必须提问

Step 3: 验证检查
  ├── 假设检查 → 隐式假设?
  ├── 委派检查:
  │   1. 有专业 Agent 匹配?
  │   2. 有合适的 task category? 有 skills 可用?
  │   3. 真的只有自己做最好?
  └── 默认偏向: 委派。只在超简单时自己做。
```

Phase 1 — 代码库评估（开放性任务）

```
Quick Assessment:
  1. 检查配置文件（linter, formatter, type config）
  2. 采样 2–3 相似文件看一致性
  3. 注意项目年龄信号

State Classification:
  ├── Disciplined → 严格遵循现有风格
  ├── Transitional → 询问遵循哪种模式
  ├── Legacy/Chaotic → 提议新约定
  └── Greenfield → 应用现代最佳实践
```

Phase 2A — 探索与研究

工具与 Agent 选择优先级：

```
免费工具（grep, glob, lsp_*, ast_grep）→ 首选
explore agent   （CHEAP）  → 代码库搜索
librarian agent（CHEAP）  → 外部文档搜索
oracle agent    （EXPENSIVE）→ 架构咨询
```

关键规则：

- Explore/Librarian **始终** `run_in_background=true`，**始终** 并行
- 并行启动 2-5 个搜索 Agent
- 搜索停止条件：足够上下文、信息重复、2 轮无新数据、直接找到答案

Phase 2B — 实施

```
Pre-Implementation:
  0. 查找并加载相关 skills
  1. 2+ 步骤 → 立即创建 Todo 列表
  2. 标记当前 in_progress
  3. 完成即刻标记 completed

Delegation Prompt 结构（强制 6 节）:
  1. TASK: 原子化具体目标
  2. EXPECTED OUTCOME: 可交付物 + 成功标准
  3. REQUIRED TOOLS: 显式工具白名单
  4. MUST DO: 穷举要求
  5. MUST NOT DO: 预防性禁止
  6. CONTEXT: 文件路径、模式、约束

Session 续传:
  - 失败/不完整 → session_id 续传
  - 后续问题 → session_id
  - 同 Agent 多轮 → 必须 session_id
  - 节省 70%+ tokens
```

Phase 2C — 失败恢复

3 次连续失败 → STOP → REVERT → DOCUMENT → 咨询 Oracle → 问用户

Phase 3 — 完成

所有 Todo 完成 + 诊断干净 + 构建通过 + 原始请求已满足

动态 Prompt 构建

> 源文件: `src/agents/sisyphus.ts` (第 152-180 行)

Sisyphus 的 prompt 不是静态文本，而是由 `buildDynamicSisyphusPrompt()` 动态组装：

```
// src/agents/sisyphus.ts
function buildDynamicSisyphusPrompt(
  model: string,
  availableAgents: AvailableAgent[],
  availableTools: AvailableTool[] = [],
  availableSkills: AvailableSkill[] = [],
  availableCategories: AvailableCategory[] = [],
  useTaskSystem = false,
): string {
  const keyTriggers = buildKeyTriggersSection(availableAgents, availableSkills);
  const toolSelection = buildToolSelectionTable(availableAgents, availableTools, availa
  const exploreSection = buildExploreSection(availableAgents);
  const librarianSection = buildLibrarianSection(availableAgents);
  const categorySkillsGuide = buildCategorySkillsDelegationGuide(availableCategories, a
  const delegationTable = buildDelegationTable(availableAgents);
  const oracleSection = buildOracleSection(availableAgents);
  const hardBlocks = buildHardBlocksSection();
  const antiPatterns = buildAntiPatternsSection();
  const deepParallelSection = buildDeepParallelSection(model, availableCategories);
  const taskManagementSection = buildTaskManagementSection(useTaskSystem);

  return `<Role>
You are "Sisyphus" - Powerful AI Agent with orchestration capabilities from OhMyOpenCod

**Why Sisyphus?**: Humans roll their boulder every day. So do you.
**Identity**: SF Bay Area engineer. Work, delegate, verify, ship. No AI slop.
**Core Competencies**:
- Parsing implicit requirements from explicit requests
- Adapting to codebase maturity (disciplined vs chaotic)
- Delegating specialized work to the right subagents
- Parallel execution for maximum throughput
</Role>
<Behavior_Instructions>
${keyTriggers}
${toolSelection}
${exploreSection}
${librarianSection}
${categorySkillsGuide}
${delegationTable}
${oracleSection}
${taskManagementSection}
${hardBlocks}
${antiPatterns}
```

```
${deepParallelSection}
</Behavior_Instructions>`;
}
```

> 源文件: `src/agents/dynamic-agent-prompt-builder.ts` — 提供以上所有 `buildXXXSection()` 函数

```ts
// src/agents/dynamic-agent-prompt-builder.ts — 核心类型
export interface AvailableAgent {
  name: string
  description: string
  metadata: AgentPromptMetadata
}

export interface AvailableTool {
  name: string
  category: "lsp" | "ast" | "search" | "session" | "command" | "other"
}

export interface AvailableSkill {
  name: string
  description: string
  location: "user" | "project" | "plugin"
}

export interface AvailableCategory {
  name: string
  description: string
  model?: string
}

// Key Triggers 构建示例
export function buildKeyTriggersSection(agents: AvailableAgent[], _skills: AvailableSki
  const keyTriggers = agents
    .filter((a) => a.metadata.keyTrigger)
    .map((a) => `- ${a.metadata.keyTrigger}`)
  if (keyTriggers.length === 0) return ""
  return `### Key Triggers (check BEFORE classification):\n\n${keyTriggers.join("\n")}\
}

// Tool Selection 表按成本排序
export function buildToolSelectionTable(
  agents: AvailableAgent[], tools: AvailableTool[] = [], _skills: AvailableSkill[] = []
): string {
  const costOrder = { FREE: 0, CHEAP: 1, EXPENSIVE: 2 }
  const sortedAgents = [...agents]
    .filter((a) => a.metadata.category !== "utility")
    .sort((a, b) => costOrder[a.metadata.cost] - costOrder[b.metadata.cost])
```

```
  // ... 构建成本排序表
}
```

Gemini 特殊处理

当 Sisyphus 使用 Gemini 模型时，注入额外的 overlay：

```
if (isGeminiModel(model)) {
  prompt += buildGeminiToolMandate()         // 强制工具使用
  prompt += buildGeminiDelegationOverride() // 委派强制
  prompt += buildGeminiVerificationOverride() // 验证强制
  prompt += buildGeminiIntentGateEnforcement() // 意图门强制
}
```

2. Hephaestus — 自主深度工作者

属性	值
源文件	`src/agents/hephaestus.ts` (539 行)
模式	`all`
模型	GPT-5.3-Codex (medium) → GPT-5.2 (medium)
角色	受 AmpCode deep mode 启发，自主目标执行者

> 源文件: `src/agents/hephaestus.ts` (第 93-103 行)

```
// src/agents/hephaestus.ts
/**
 * Hephaestus - The Autonomous Deep Worker
 *
 * Named after the Greek god of forge, fire, metalworking, and craftsmanship.
 * Inspired by AmpCode's deep mode - autonomous problem-solving with thorough research.
 *
 * Powered by GPT Codex models.
 * Optimized for:
 * - Goal-oriented autonomous execution (not step-by-step instructions)
 */
const MODE: AgentMode = "all"
```

核心特点

- **NEVER ask permission** — 禁止任何形式的请求确认
- **100% 完成或不做** — 部分实现 = 失败
- **"Senior Staff Engineer"** — 不猜测，验证；不早停，完成
- **被阻塞时**：尝试不同方法 → 分解问题 → 挑战假设 → 借鉴他人方案
- **问用户是最后手段**

Todo 纪律 (Hephaestus 特色)

```
// src/agents/hephaestus.ts - Todo/Task discipline section
function buildTodoDisciplineSection(useTaskSystem: boolean): string {
  if (useTaskSystem) {
    return `## Task Discipline (NON-NEGOTIABLE)
**Track ALL multi-step work with tasks. This is your execution backbone.**
### Workflow (STRICT)
1. **On task start**: task_create with atomic steps
2. **Before each step**: task_update(status="in_progress") (ONE at a time)
3. **After each step**: task_update(status="completed") IMMEDIATELY (NEVER batch)
4. **Scope changes**: Update tasks BEFORE proceeding
**NO TASKS ON MULTI-STEP WORK = INCOMPLETE WORK.**`;
  }
  // fallback 到 TodoWrite 系统 ...
}
```

意图提取（比 Sisyphus 更激进）

表面形式	真实意图	Hephaestus 的响应
"Did you do X?" (没做)	你忘了。现在做。	承认 → 立即做
"How does X work?"	理解 X 以便修复	探索 → 实现/修复
"What's the best way to Z?"	实际做 Z	决定 → 实现
"Why is A broken?"	修复 A	诊断 → 修复

默认：消息隐含行动，除非用户显式说"只解释"

3. Atlas — 编排执行器

属性	值
源文件	`src/agents/atlas/` (6 个文件)
模式	`all`
模型	Kimi K2.5 → Claude Sonnet 4-6 → GPT-5.2
角色	"交响乐指挥" — 编排 task() 完成 Todo 列表

模型路由

```
function getAtlasPromptSource(model?: string): AtlasPromptSource {
  if (isGptModel(model))    return "gpt"     // → gpt.ts
  if (isGeminiModel(model)) return "gemini"   // → gemini.ts
  return "default"                            // → default.ts (Claude)
}
```

Prompt 动态注入

Atlas 的 prompt 通过模板替换注入动态内容：

```
basePrompt
  .replace("{CATEGORY_SECTION}", categorySection)
  .replace("{AGENT_SECTION}", agentSection)
  .replace("{DECISION_MATRIX}", decisionMatrix)
  .replace("{SKILLS_SECTION}", skillsSection)
  .replace("{{CATEGORY_SKILLS_DELEGATION_GUIDE}}", categorySkillsGuide)
```

`prompt-section-builder.ts` 提供构建这些节的工具函数。

4. Prometheus — 计划生成器

属性	值
源文件	`src/agents/prometheus/` (8 个文件)
模式	`subagent`
模型	Claude Opus 4-6 (max) → GPT-5.2 (high) → Kimi K2.5 → Gemini 3.1 Pro
角色	纯规划者，绝不写代码

文件结构

文件	职责
`system-prompt.ts`	组装最终 prompt（按模型分发 Claude/GPT/Gemini）
`identity-constraints.ts`	核心身份约束：纯规划者
`interview-mode.ts`	Phase 1：意图分类 + 分策略面谈
`plan-generation.ts`	Phase 2：Metis 咨询 → 生成计划 → 自审 → 差距处理
`high-accuracy-mode.ts`	Phase 3：Momus 审核循环
`plan-template.ts`	计划文件 Markdown 模板
`behavioral-summary.ts`	阶段总结 + 草稿清理
`gpt.ts` / `gemini.ts`	模型优化变体

三阶段工作流

详见 03-plan-build-mode.md

关键约束

- **仅写** `.sisyphus/plans/*.md` 和 `.sisyphus/drafts/*.md`（由 `prometheus-md-only` hook 强制）
- 所有请求均解读为"制定计划"，不执行
- 面谈每轮自动清关检查（6 项全 YES 自动进入计划生成）
- 单一计划原则：所有内容放入**一个**计划文件
- 零人工干预原则：所有验收标准必须是 Agent 可执行的命令

5. Metis — 计划前分析师

属性	值
源文件	`src/agents/metis.ts`
模式	`subagent`
模型	Claude Opus 4-6 (max) → Kimi K2.5 → GPT-5.2 → Gemini 3.1 Pro
温度	0.3
Thinking	32k tokens
角色	在 Prometheus 规划前预分析请求

6 种意图分类及差异化分析

意图类型	分析焦点	预研工具
Refactoring	行为保持、回滚策略、影响范围	`lsp_find_references`，`ast_grep_search`
Build from Scratch	先发现现有模式再提问	`explore` + `librarian` 子代理
Mid-sized Task	精确界限、AI-slop 防御	无预研

意图类型	分析焦点	预研工具
Collaborative	频繁检查点、假设列表	—
Architecture	战略分析、约束识别	`oracle` 咨询
Research	退出标准、并行探查策略	`explore` + `librarian` 并行

输出格式

```
## Intent Classification
[类型 + 信心度 + 理由]

## Pre-Analysis Findings
[工具调用结果总结]

## Questions for User
[澄清问题列表]

## Risk Identification
[风险 + 缓解策略]

## Prometheus Instructions
[给 Prometheus 的精确指令]
```

工具限制

禁止 `write`，`edit`，`apply_patch`，`task` — 纯只读分析

6. Oracle — 战略顾问

属性	值
源文件	`src/agents/oracle.ts` (171 行)
模式	`subagent`
模型	GPT-5.2 (high) → Gemini 3.1 Pro → Claude Opus 4-6 (max)

属性	值
温度	0.1
角色	复杂架构设计、深度调试、安全/性能评审

> 源文件: `src/agents/oracle.ts`

```
// src/agents/oracle.ts — Prompt 元数据 & 系统 prompt
const MODE: AgentMode = "subagent"

export const ORACLE_PROMPT_METADATA: AgentPromptMetadata = {
  category: "advisor",
  cost: "EXPENSIVE",
  promptAlias: "Oracle",
  triggers: [
    { domain: "Architecture decisions", trigger: "Multi-system tradeoffs, unfamiliar pa
    { domain: "Self-review", trigger: "After completing significant implementation" },
    { domain: "Hard debugging", trigger: "After 2+ failed fix attempts" },
  ],
  useWhen: [
    "Complex architecture design",
    "After completing significant work",
    "2+ failed fix attempts",
    "Unfamiliar code patterns",
    "Security/performance concerns",
    "Multi-system tradeoffs",
  ],
  avoidWhen: [
    "Simple file operations (use direct tools)",
    "First attempt at any fix (try yourself first)",
    "Questions answerable from code you've read",
    "Trivial decisions (variable names, formatting)",
    "Things you can infer from existing code patterns",
  ],
}

const ORACLE_SYSTEM_PROMPT = `You are a strategic technical advisor with deep reasoning

<decision_framework>
Apply pragmatic minimalism in all recommendations:
- **Bias toward simplicity**: The right solution is typically the least complex one.
- **Leverage what exists**: Favor modifications to current code over new components.
- **One clear path**: Present a single primary recommendation.
- **Signal the investment**: Quick(<1h), Short(1-4h), Medium(1-2d), Large(3d+).
- **Know when to stop**: "Working well" beats "theoretically optimal."
</decision_framework>

<output_verbosity_spec>
- **Bottom line**: 2-3 sentences maximum. No preamble.
- **Action plan**: ≤7 numbered steps. Each step ≤2 sentences.
```

```
- **Why this approach**: ≤4 bullets when included.
- **Watch out for**: ≤3 bullets when included.
</output_verbosity_spec>`
```

工具限制

```
// src/agents/oracle.ts — 只读工具限制
const restrictions = createAgentToolRestrictions([
  "write", "edit", "apply_patch", "task", "call_omo_agent",
])
```

关键约束

- **只读** — 禁止 write/edit/apply_patch/task
- Sisyphus **必须等待 Oracle 结果**才能给出最终答案
- **NEVER cancel Oracle**
- **NEVER** `background_cancel(all=true)` 当 Oracle 运行时

7. Explore — 代码库搜索专家

属性	值
源文件	`src/agents/explore.ts` (123 行)
模式	`subagent`
模型	Grok Code Fast → MiniMax M2.5 → Claude Haiku 4-5 → GPT-5 Nano
角色	"Contextual Grep" — 内部代码搜索
成本	**FREE** — 极低成本模型

> 源文件: `src/agents/explore.ts`

```
// src/agents/explore.ts — 完整工厂函数
const MODE: AgentMode = "subagent"

export const EXPLORE_PROMPT_METADATA: AgentPromptMetadata = {
  category: "exploration",
  cost: "FREE",
  promptAlias: "Explore",
  keyTrigger: "2+ modules involved → fire `explore` background",
  triggers: [
    { domain: "Explore", trigger: "Find existing codebase structure, patterns and style
  ],
  useWhen: [
    "Multiple search angles needed",
    "Unfamiliar module structure",
    "Cross-layer pattern discovery",
  ],
  avoidWhen: [
    "You know exactly what to search",
    "Single keyword/pattern suffices",
    "Known file location",
  ],
}

export function createExploreAgent(model: string): AgentConfig {
  const restrictions = createAgentToolRestrictions([
    "write", "edit", "apply_patch", "task", "call_omo_agent",
  ])

  return {
    description: 'Contextual grep for codebases. Answers "Where is X?", "Which file has
    mode: MODE,
    model,
    temperature: 0.1,
    ...restrictions,
    prompt: `You are a codebase search specialist. Your job: find files and code, retur

## CRITICAL: What You Must Deliver

### 1. Intent Analysis (Required)
Before ANY search, wrap your analysis in <analysis> tags.

### 2. Parallel Execution (Required)
Launch **3+ tools simultaneously** in your first action.
```

```
### 3. Structured Results (Required)
Always end with <results><files>...</files><answer>...</answer></results>`,
  }
}
```

8. Librarian — 外部知识专家

属性	值
源文件	`src/agents/librarian.ts` (321 行)
模式	`subagent`
模型	Gemini 3 Flash → MiniMax M2.5 → Big Pickle
角色	"Reference Grep" — 外部文档搜索
成本	**CHEAP**
工具	context7, web search, grep_app, gh CLI

> 源文件: `src/agents/librarian.ts`

```
// src/agents/librarian.ts — Prompt 元数据 & 工厂函数
export const LIBRARIAN_PROMPT_METADATA: AgentPromptMetadata = {
  category: "exploration",
  cost: "CHEAP",
  promptAlias: "Librarian",
  keyTrigger: "External library/source mentioned → fire `librarian` background",
  triggers: [
    { domain: "Librarian", trigger: "Unfamiliar packages / libraries, struggles at weir
  ],
  useWhen: [
    "How do I use [library]?",
    "What's the best practice for [framework feature]?",
    "Why does [external dependency] behave this way?",
    "Working with unfamiliar npm/pip/cargo packages",
  ],
}

export function createLibrarianAgent(model: string): AgentConfig {
  const restrictions = createAgentToolRestrictions([
    "write", "edit", "apply_patch", "task", "call_omo_agent",
  ])

  return {
    description: "Specialized codebase understanding agent for multi-repository analysi
    mode: MODE,
    model,
    temperature: 0.1,
    ...restrictions,
    prompt: `# THE LIBRARIAN
You are **THE LIBRARIAN**, a specialized open-source codebase understanding agent.

## PHASE 0: REQUEST CLASSIFICATION
Classify EVERY request into one of these categories:
- **TYPE A: CONCEPTUAL**: "How do I use X?" → Doc Discovery → context7 + websearch
- **TYPE B: IMPLEMENTATION**: "How does X implement Y?" → gh clone + read + blame
- **TYPE C: CONTEXT**: "Why was this changed?" → gh issues/prs + git log/blame
- **TYPE D: COMPREHENSIVE**: Complex/ambiguous → ALL tools`,
  }
}
```

四种请求分类

类型	描述	策略
TYPE A	概念性	文档发现 → context7 + websearch
TYPE B	实现细节	`gh repo clone` + read + blame
TYPE C	上下文信息	gh issues/PRs + git log/blame
TYPE D	综合性	全工具链

9. Momus — 计划审查员

属性	值
源文件	`src/agents/momus.ts` (244 行)
模式	`subagent`
模型	GPT-5.2 (medium) → Claude Opus 4-6 → Gemini 3.1 Pro
温度	0.1
角色	审查 `.sisyphus/plans/*.md`，只找阻塞性问题

> 源文件: `src/agents/momus.ts`

```
// src/agents/momus.ts — 核心系统 prompt
/**
 * Momus - Plan Reviewer Agent
 *
 * Named after Momus, the Greek god of satire and mockery, who was known for
 * finding fault in everything - even the works of the gods themselves.
 */
const MODE: AgentMode = "subagent"

export const MOMUS_SYSTEM_PROMPT = `You are a **practical** work plan reviewer.
Your goal is simple: verify that the plan is **executable** and **references are valid**

## Your Purpose
You exist to answer ONE question: "Can a capable developer execute this plan without ge

You are NOT here to:
- Nitpick every detail
- Demand perfection
- Question the author's approach or architecture choices

**APPROVAL BIAS**: When in doubt, APPROVE. A plan that's 80% clear is good enough.

## What You Check (ONLY THESE)

### 1. Reference Verification (CRITICAL)
- Do referenced files exist?
- Do referenced line numbers contain relevant code?
**FAIL only if**: Reference doesn't exist OR points to completely wrong content.

### 2. Executability Check (PRACTICAL)
- Can a developer START working on each task?
**FAIL only if**: Task is so vague that developer has NO idea where to begin.

### 3. Critical Blockers Only
- Missing information that would COMPLETELY STOP work
**NOT blockers**: Missing edge case handling, incomplete acceptance criteria, stylistic
`
```

判定输出

- `[OKAY]` — 默认判定
- `[REJECT]` — 最多 3 个阻塞性 issue

10. Multimodal-Looker — 多模态查看器

属性	值
源文件	`src/agents/multimodal-looker.ts`
模式	`subagent`
角色	视觉内容分析

11. Sisyphus-Junior — 工作执行器

属性	值
源文件	`src/tools/delegate-task/sisyphus-junior-agent.ts`
角色	`task(category="...")` 时自动生成的执行器

不是独立 Agent 定义。当调用 `task(category="quick", ...)` 时，系统自动创建一个 Sisyphus-Junior Agent，使用该 category 对应的模型，并注入 category 专属的 prompt append。

```
export const SISYPHUS_JUNIOR_AGENT = getAgentDisplayName("sisyphus-junior")
```

Agent 元数据系统

每个 Agent 携带 `AgentPromptMetadata`，供 Sisyphus 动态构建 prompt：

```ts
interface AgentPromptMetadata {
  category: "exploration" | "specialist" | "advisor" | "utility"
  cost: "FREE" | "CHEAP" | "EXPENSIVE"
  triggers: DelegationTrigger[]      // 域 → Agent 映射
  useWhen?: string[]                 // 使用场景
  avoidWhen?: string[]               // 避免场景
  dedicatedSection?: string          // 专属 prompt 节
  promptAlias?: string               // Prompt 中的别名
  keyTrigger?: string                // Phase 0 触发条件
}
```

模型 Fallback 链总览

Agent	首选模型	Fallback 1	Fallback 2	Fallback 3
Sisyphus	claude-opus-4-6 (max)	kimi-k2.5-free	glm-5	big-pickle
Hephaestus	gpt-5.3-codex (medium)	gpt-5.2 (copilot)	—	—
Atlas	kimi-k2.5-free	claude-sonnet-4-6	gpt-5.2	—
Prometheus	claude-opus-4-6 (max)	gpt-5.2 (high)	kimi-k2.5-free	gemini-3.1-pro
Metis	claude-opus-4-6 (max)	kimi-k2.5-free	gpt-5.2 (high)	gemini-3.1-pro
Momus	gpt-5.2 (medium)	claude-opus-4-6 (max)	gemini-3.1-pro	—
Oracle	gpt-5.2 (high)	gemini-3.1-pro	claude-opus-4-6 (max)	—
Explore	grok-code-fast-1	minimax-m2.5-free	claude-haiku-4-5	gpt-5-nano
Librarian	gemini-3-flash	minimax-m2.5-free	big-pickle	—

模型解析管线：**3 步**

1. 用户覆盖（override）→ 2. 约束过滤（availableModels）→ 3. Fallback + 系统默认